Menos *macroblocking* en video en vivo sobre SRT: *I-headers* entregados por el edge

Vicente Andrade Ercilla
Estudiante de Ciencias de la Ingeniería Electrónica
Departamento de Electrónica, Universidad Técnica Federico Santa María, Valparaíso, Chile
vicente.andrade@usm.cl

Resumen

En video en vivo sobre UDP (*User Datagram Protocol*), un paquete perdido produce *macroblocking*: bloques corruptos que se propagan hasta el próximo cuadro intra, que con un GoP (*Group of Pictures*) de 90 cuadros a 30 fps llega cada 3 s. SRT (*Secure Reliable Transport*) retransmite, pero solo dentro de su latencia; lo que no llega a tiempo se descarta. Proponemos que el *edge*, el servidor que reparte el video, reutilice los *I-headers*, copias intra de algunos cuadros predichos (P) que el codificador ya le envía para que los espectadores nuevos no esperen el próximo intra, y los entregue solo al espectador que descartó paquetes. No se pide nada al codificador ni se agrega latencia. El *edge* detecta el descarte con la marca de tiempo de los ACK (*acknowledgement*) de SRT. Emulamos todo en Linux: el video se codifica con libx264, se decodifica con el decodificador H.264 de FFmpeg y los artefactos se miden con SSIM (*Structural Similarity*) contra el video original. Con la latencia por defecto de SRT, en las corridas con cortes que SRT no recupera, el *I-header* acorta los artefactos visibles en un tercio con un GoP de 90 cuadros (de 1,55 a 1,0 s) y en un 15 % con un GoP de 30, ambos a 30 fps. Con cortes cortos, que SRT ya recupera, no mejora. El costo es un 67 % más de tasa de bits entre el codificador y el *edge* (de 3,1 a 5,1 Mbps), un cuadro hasta 9 veces más grande que un P al entregar el *I-header* y una pequeña diferencia con la imagen correcta hasta el próximo intra, porque los P siguientes se codificaron sobre el P original y no sobre el *I-header*.

Palabras clave: H.264, SRT, pérdida de paquetes, *macroblocking*, *edge*, GoP.

I. Introducción

En video en vivo, el codificador intercala cuadros intra (I), que se decodifican solos, y cuadros predichos (P), que solo describen lo que cambió respecto del anterior. Un GoP largo ahorra bits, pero si se pierde un paquete, el error se copia de P en P hasta el próximo intra y el espectador ve *macroblocking*. Sobre TCP (*Transmission Control Protocol*) la pérdida se convierte en pausas; este trabajo trata el transporte sobre UDP, que se usa cuando importa la latencia.

SRT [1] retransmite lo perdido, pero entrega cada paquete con una latencia fija L. Si un corte de la red dura más que L, el paquete se descarta y el *macroblocking* vuelve. Las salidas conocidas les cobran a todos los espectadores: subir L retrasa a todos, acortar el GoP sube la tasa de bits de todos y pedirle un intra al codificador con un PLI (*Picture Loss Indication*) se lo envía a todos.

AGiLE [2] envía al *edge*, el servidor que reparte el video a los espectadores, cada cierto número de P, un *I-header*: el mismo P codificado además como intra, dentro de una SEI (*Supplemental Enhancement Information*). Lo usa para que quien entra al canal no espere el próximo intra. Para el resto, ese *I-header* ya está en el *edge* y se descarta.

Objetivo. Que el espectador que perdió paquetes salga del *macroblocking* en el próximo *I-header*, sin pedir nada al codificador, sin agregar latencia y sin cambiar lo que recibe el espectador sin pérdidas. Las contribuciones son tres:

1. El *edge* le entrega el *I-header* solo al espectador que descartó paquetes (Sección V).
2. Una regla para que el emisor SRT sepa qué descartó el receptor, solo con la marca de tiempo de los ACK (*acknowledgement*) (Sección V-A).
3. Un emulador en C++ sobre Linux y una comparación de UDP y SRT con y sin *I-header*, con GoP de 90 y 30 cuadros, que incluye lo que cuesta el *I-header* (Sección VI).

La Sección II resume lo necesario de H.264 y SRT, y la Sección III los trabajos relacionados. La Sección IV plantea el problema, la Sección V la estrategia y la arquitectura, la Sección VI los resultados y la Sección VII las conclusiones.

II. Antecedentes

A. H.264

En H.264 [3], cada imagen se divide en macrobloques de 16×16 píxeles, agrupados en *slices*, y cada *slice* viaja en una NAL (*Network Abstraction Layer unit*). Un P copia cada macrobloque desde el cuadro anterior, desplazado según un vector de movimiento; si ese cuadro está dañado, la copia también. Un IDR (*Instantaneous Decoding Refresh*) es un intra que además vacía las referencias del decodificador. El *I-header* de AGiLE [2] es un intra no-IDR guardado en una SEI de datos de usuario, que el decodificador ignora si no la conoce. El *I-header* muestra la misma imagen que el P que reemplaza, pero no es idéntico bit a bit, porque está codificado

de otra forma. Los P siguientes describen cambios respecto del P original; aplicados sobre el *I-header*, dan una imagen levemente distinta de la correcta, y esa diferencia se arrastra hasta el próximo intra. A esa diferencia la llamamos deriva. La SEI *recovery point* le indica al decodificador que puede retomar desde ese intra.

B. *SRT en modo* live

El video viaja en paquetes TS (*Transport Stream* de MPEG, *Moving Picture Experts Group*) de 188 B, siete por cada paquete SRT. El receptor de SRT [1] avisa cada hueco de inmediato con un NAK (*negative acknowledgement*) y lo repite periódicamente; el emisor retransmite con prioridad. Cada 10 ms, el receptor envía un ACK con el primer paquete que le falta. TSBPD (*Timestamp-Based Packet Delivery*) entrega cada paquete a la aplicación en su marca de tiempo más L, que por defecto es de 120 ms [1]. Si un paquete todavía falta en ese momento, TLPKTDROP (*Too-Late Packet Drop*) lo salta y el ACK siguiente lo cubre como si hubiera llegado. Según el borrador, el emisor no se entera de ese salto [1].

III. Trabajos relacionados

AGiLE [2] usa un GoP de 10 s con un *I-header* cada 2 s, que el *edge* solo entrega a quien entra al canal. Frente a un GoP fijo de 2 s, logra un *BD-rate* (*Bjøntegaard Delta rate*, diferencia promedio de bits a igual calidad) de $-3{,}5$ %. Entrar por un *I-header* cuesta 1,1 puntos de VMAF (*Video Multimethod Assessment Fusion*), por la deriva. AGiLE no trata las pérdidas.

Keyframe injection [4, 5] mantiene un flujo aparte de intras, del que el reproductor pide uno al entrar o al perder paquetes, con una caída menor que 0,94 puntos de VMAF [4]. El estudio se extendió a tres generaciones de estándares de video [5], y una versión posterior repara con un P apoyado en un cuadro anterior a la pérdida, para evitar picos de tasa [6].

FrameTrace [7] mide en *Media over QUIC* la fracción de cuadros que se muestran. Con 1 % de pérdida informa VMAF 72,6 con GoP de 0,5 s y 55,7 con GoP de 2 s, y acorta el GoP cuando hay pérdida, lo que exige un lazo hasta el codificador, igual que el PLI de RTP (*Real-time Transport Protocol*).

En los trabajos revisados, la reparación sale de un flujo aparte o de un pedido al codificador. Aquí sale de un intra que ya está en el *edge*, y la decisión la toma el *edge*, sin un pedido del reproductor.

IV. Problema

Sean t_0 el instante en que el *edge* envía un paquete, d el retardo de ida y D la duración de un corte. El receptor solo puede pedir el paquete cuando el corte termina, y la retransmisión debe llegar antes del plazo $t_0 + d + L$. En primera aproximación, SRT recupera el corte si

$$D + \mathrm{RTT} + d \lesssim L, \tag{1}$$

donde RTT es el tiempo de ida y vuelta. Con $L = 120$ ms, $d = 15$ ms y RTT $= 30$ ms, SRT recupera cortes de hasta unos 75 ms. Haivision recomienda L entre 3 y 6 RTT para pérdidas de hasta 7 % [8], es decir, de 90 a 180 ms en este caso. Sea cual sea L, los cortes más largos que $L - \mathrm{RTT} - d$ se descartan: el *slice* llega truncado y el *macroblocking* dura hasta el próximo intra (Fig. 1).

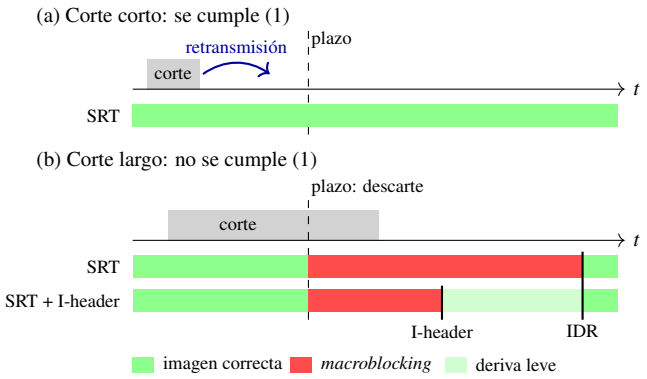


Fig. 1. Qué ve el espectador según la duración del corte. La barra gris es el corte de la red y la línea discontinua el plazo de entrega del primer paquete perdido. (a) La retransmisión llega antes del plazo y la imagen no se daña. (b) El paquete se descarta en su plazo. Con SRT solo, el *macroblocking* dura hasta el IDR; con *I-header*, hasta el primer *I-header* que el *edge* entrega después de detectar el descarte, seguido de una deriva leve hasta el IDR.

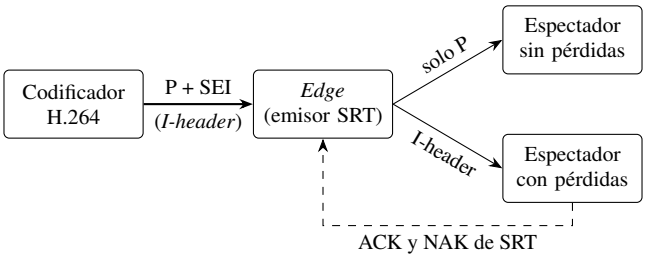


Fig. 2. Arquitectura. El *I-header* viaja siempre hasta el *edge*, que lo entrega solo al espectador que descartó paquetes. El *edge* lo sabe por los ACK y NAK de SRT, sin un mensaje nuevo.

Problema. Acortar el *macroblocking* de los cortes que SRT no recupera, en el tramo *edge*–espectador, con tres condiciones: no agregar latencia, no cambiar lo que recibe el espectador sin pérdidas y no pedir nada al codificador por cada pérdida. Suponemos que el tramo codificador–*edge* no pierde paquetes.

V. Estrategia y arquitectura

La Fig. 2 muestra la arquitectura. El codificador envía al *edge* el *stream* con GoP largo y, cada n P, un *I-header* dentro de una SEI. El *edge* abre una conexión SRT con cada espectador. Al espectador sin pérdidas le quita las SEI, así que recibe exactamente el *stream* original. Cuando detecta que un espectador descartó paquetes, reemplaza el próximo P que tenga *I-header* por la SEI *recovery point* y el *I-header*, solo para ese espectador.

A. *Cómo detecta el* edge *el descarte*

El *edge* no recibe aviso del descarte, pero lo puede inferir. El receptor envía un ACK cada $T_{\mathrm{ACK}} = 10$ ms, y si salta un paquete, el ACK siguiente lo cubre igual. Ese ACK lleva la marca de tiempo de cuando se generó [1]. Un paquete s que el *edge* vio en un NAK se descartó si el primer ACK que lo cubre se generó después de su plazo y después de cuando se habría confirmado

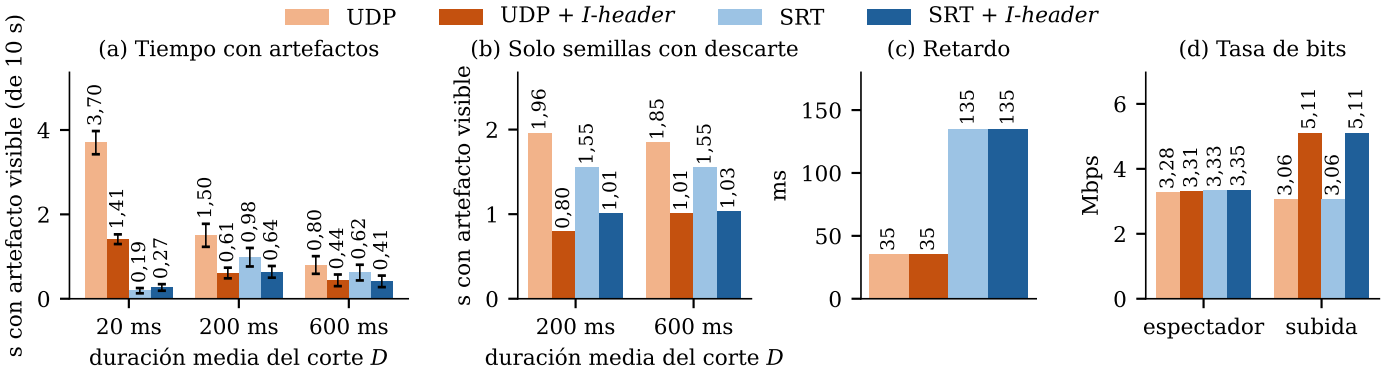


Fig. 3. Los cuatro casos con GoP 90 a 30 fps y $L = 120$ ms. Media de 30 semillas de 10 s; los cuatro casos ven los mismos cortes. (a) Segundos, de los 10, con artefacto visible (ΔSSIM $> 0{,}05$) según la duración media del corte D, con su error estándar. (b) Lo mismo, solo en las semillas donde el caso sin *I-header* tuvo artefactos ($D = 200$ ms: 23 con UDP y 19 con SRT; $D = 600$ ms: 13 y 12). (c) Retardo extremo a extremo: ida (15 ms) más búfer de UDP (20 ms) o latencia de SRT (120 ms). (d) Tasa de bits media hacia el espectador ($D = 200$ ms) y de subida codificador–*edge*.

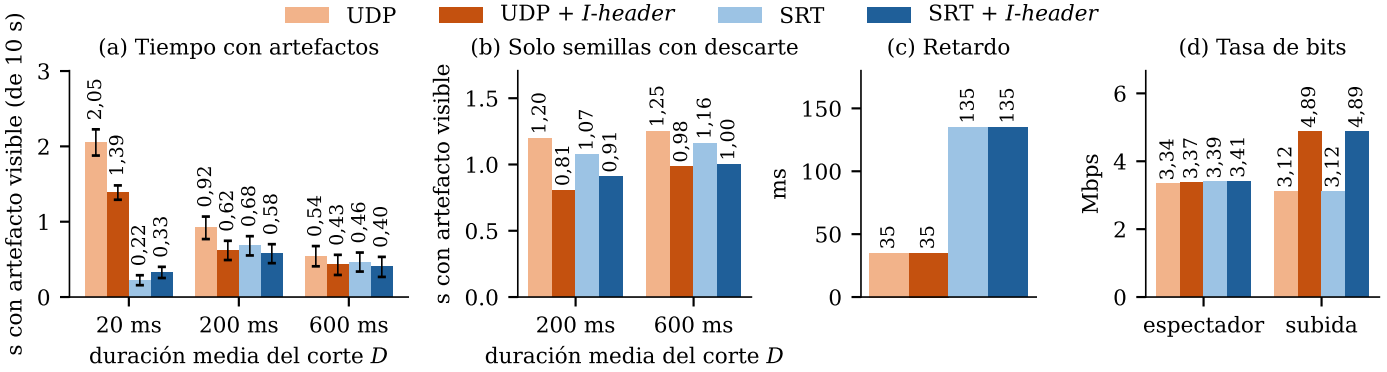


Fig. 4. Igual que la Fig. 3, con GoP 30 (un intra por segundo). En (b) las semillas son las mismas: 23 y 19 con $D = 200$ ms, 13 y 12 con $D = 600$ ms.

su última retransmisión:

$$g_s > \max\left(\tau_s,\ r_s + d + T_{\mathrm{ACK}}\right), \tag{2}$$

donde g_s es el instante de generación de ese ACK, $\tau_s = t_{0,s} + d + L$ el plazo de s y r_s el instante de su última retransmisión (si no hubo, solo cuenta τ_s). El segundo término evita marcar un paquete recuperado justo antes de su plazo. La regla supone que el *edge* no envía retransmisiones que ya no llegarían a tiempo.

B. Generación del I-header

El *I-header* es un intra no-IDR con el número de cuadro del P que reemplaza. libx264 no emite intras no-IDR sin cuadros B, así que el emulador codifica la misma secuencia con un intra cada 5 cuadros, a varios QP (parámetro de cuantización) fijos; para cada P toma el intra de QP más cercano y reescribe su encabezado de IDR como intra no-IDR. El *I-header* debe tener el QP del P: con un *I-header* de calidad fija, la deriva fue grande donde el codificador tenía pocos bits.

C. Emulador y métricas

Todo se emuló en un PC con Linux (Ubuntu 24.04). Un programa en C++17 lee el H.264, arma la SEI, reescribe el encabezado y genera el TS, y un simulador de eventos discretos modela SRT en modo *live* según el borrador [1]. El canal alterna entre un estado bueno y cortes sin entrega de duración media D (modelo de Gilbert–Elliott) y depende solo del tiempo y de una semilla, así que todos los casos ven los mismos cortes. Sin pérdida, el video decodificado es idéntico bit a bit al original. El video se codifica con libx264 (versión 164) y se decodifica con el decodificador H.264 de FFmpeg 6.1, con su ocultamiento de errores; solo la red y SRT se simulan. Un cuadro que no llega se reemplaza por el anterior.

Métrica. Medimos lo que ve el espectador con SSIM (*Structural Similarity*) [9], que compara la estructura local de dos imágenes en vez de su diferencia píxel a píxel. En cada ventana gaussiana de 11×11 píxeles de la luminancia,

$$\mathrm{SSIM}(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \tag{3}$$

donde μ, σ^2 y σ_{xy} son las medias, varianzas y covarianza locales, y C_1, C_2 constantes que evitan dividir por cero. El SSIM del cuadro es el promedio de las ventanas (1 si las imágenes son iguales). Lo implementamos como lo describen los autores, con la imagen promediada en bloques de 3×3. Preferimos SSIM a VMAF porque su cálculo es explícito y se puede verificar. Para cada cuadro k,

$$\Delta\mathrm{SSIM}_k = \mathrm{SSIM}(F_k, S_k) - \mathrm{SSIM}(F_k, R_k), \tag{4}$$

donde F_k es el cuadro original (la fuente), S_k el cuadro decodificado sin pérdida y R_k el recibido. Al comparar ambos con la fuente, la deriva del *I-header*, que se parece al original, pesa poco, y el *macroblocking* pesa mucho. Un cuadro tiene un artefacto visible si $\Delta\text{SSIM}_k > \theta = 0{,}05$. En los cuadros dañados por pérdida, la mediana de ΔSSIM_k va de 0,15 a 0,43 según el caso. El umbral es nuestro, así que también informamos $\theta = 0{,}02$ y 0,1, y la pérdida acumulada $\sum_k \max(0, \Delta\text{SSIM}_k)/\text{fps}$, que no usa umbral.

VI. Resultados

Configuración. Los primeros 300 cuadros (10 s) de la secuencia pública *aspen* de Xiph.org [10], escalados a 1280×720 y tratados como 30 fps, codificados con libx264 en perfil *Baseline* a 3 Mbps, con 45 *slices* por cuadro y GoP de 90 o 30 cuadros. Hay un *I-header* cada 5 P, con el QP del P que reemplaza. El retardo de ida es de 15 ms y el *edge* envía a 6 Mbps como máximo. SRT usa $L = 120$ ms, su valor por defecto, o 180 ms, el que recomienda Haivision para 4 % de pérdida [8]; sin SRT, el espectador tiene un búfer de 20 ms. Elegimos cortes a ambos lados de (1): de 20 ms (2 % del tiempo), que caben en L, y de 200 y 600 ms (4 %), que no caben. No afirmamos con qué frecuencia ocurren estos cortes en una red real. Cada caso se corre con las mismas 30 semillas. Con UDP + *I-header*, el espectador avisa cada descarte; con SRT + *I-header*, el *edge* usa la regla (2). El codificador y el decodificador corren con un hilo, así que cada corrida es determinista.

A. Calidad, latencia y ancho de banda

Las Figs. 3 y 4 comparan los cuatro casos. Sin retransmisión, el *I-header* reduce el tiempo con artefacto visible con los tres tipos de corte (Figs. 3a y 4a); con GoP 90, de 3,70 a 1,41 s con cortes de 20 ms y de 1,50 a 0,61 s con cortes de 200 ms. Con cortes de 20 ms, SRT recupera casi todo (0,19 s) y el *I-header* no ayuda: el tiempo sobre el umbral sube a 0,27 s, porque cambia artefactos breves por deriva apenas sobre 0,05. La pérdida acumulada de SSIM es la misma (0,041 y 0,040), y con $\theta = 0{,}1$ el tiempo baja de 0,12 a 0,08 s.

Con cortes de 200 y 600 ms, SRT no recupera el corte en 19 y 12 de las 30 semillas. En esas semillas, con GoP 90, el *I-header* acorta (Fig. 3b) el artefacto de 1,55 a 1,01 s y de 1,55 a 1,03 s. Comparando semilla a semilla con cortes de 200 ms, mejora en 13, empeora en 4 y queda igual en 13. Con GoP 30 la ganancia es menor (de 1,07 a 0,91 s y de 1,16 a 1,00 s), porque el intra llega antes de todos modos. Con GoP 90, la ventaja se mantiene con $\theta = 0{,}02$ y 0,1; con GoP 30 y $\theta = 0{,}02$ desaparece, porque la deriva supera ese umbral. Con cortes largos, UDP + *I-header* queda cerca de SRT + *I-header* con 100 ms menos de retardo, pero con cortes cortos es mucho peor.

El *I-header* no agrega retardo (Fig. 3c). La alternativa es subir L para todos (Tabla I). Para igualar al *I-header* con $L = 120$ ms, SRT necesita entre 180 y 250 ms con cortes de 200 ms, y 500 ms con cortes de 600 ms. Con la L recomendada de 180 ms, el *I-header* sigue ayudando. La tasa de bits hacia el espectador crece 0,7 % frente a SRT (Fig. 3d), porque el *I-header* solo baja cuando hay pérdida.

Tabla I. Segundos con artefacto visible (de 10 s) con GoP 90 según la latencia L de SRT. Media de 30 semillas. +IH: con *I-header*.

	$L = 120$ ms		$L = 180$ ms		solo SRT		
D	SRT	+IH	SRT	+IH	250	500	1000 ms
200 ms	0,98	0,64	0,62	0,47	0,45	0,25	0,00
600 ms	0,62	0,41	0,52	0,39	0,47	0,40	0,14

Tabla II. Costo del *I-header* frente al mismo caso sin *I-header*.

Costo	Valor
Tasa de bits codificador–*edge*	+67 % con GoP 90 (3,06 a 5,11 Mbps); +57 % con GoP 30 (3,12 a 4,89 Mbps)
Tasa de bits al espectador	+0,7 % (SRT, $D = 200$ ms)
Retardo agregado	0
Pico al entregar	cuadro de hasta 105 kB; un P medio pesa 12 kB
Deriva hasta el intra	ΔSSIM mediana 0,006 (GoP 90) y 0,010 (GoP 30); superan 0,05 en algún cuadro 5 de 56 y 6 de 50 *I-headers*
Cortes cortos ($D = 20$ ms)	0,19 a 0,27 s sobre θ (GoP 90); igual pérdida acumulada
Errores de detección	1 *I-header* de más y 2 de menos en 90 corridas
Cómputo en codificador y *edge*	no medido

B. Detección del descarte

En las 90 corridas de SRT + *I-header* con GoP 90 y $L = 120$ ms, 46 tuvieron descartes. El *edge* envió un *I-header* en 1 corrida sin descarte y no lo envió en 2 con descarte. Por paquete, el 94 % de los marcados por (2) fueron descartes reales; los demás son paquetes recibidos justo después de un hueco descartado, cuyo ACK llega tarde. Como basta un paquete marcado por corte, ese error casi no cambia la decisión.

C. Qué perdemos

La Tabla II resume el costo del *I-header*. El mayor es la subida: un *I-header* cada 5 P lleva la tasa de bits codificador–*edge* de 3,06 a 5,11 Mbps, aunque se paga una vez y no por espectador. Al entregarlo, el cuadro con *I-header* pesa hasta 105 kB, 9 veces un P medio; a 6 Mbps tarda unos 140 ms en salir del *edge*, y el emulador lo incluye. Después de recuperar, el espectador no vuelve a la imagen exacta hasta el intra. Para medir esa deriva sola, sin pérdida, se forzó un *I-header* en cada P múltiplo de 5, uno por corrida, y se midió (4) hasta el intra siguiente. Con GoP 90, la mediana fue 0,006 y el 90 % de los cuadros quedó bajo 0,017. Con el criterio PSNR (*Peak Signal-to-Noise Ratio*) menor que 35 dB contra el *stream* sin pérdida, el 43 % de esos *I-headers* contaría como daño, aunque se vean casi igual que el original; por eso no usamos PSNR.

D. Validez

Estos resultados tienen cuatro límites. Primero, usan una sola secuencia de 10 s y un canal de dos estados. Segundo, el modelo de SRT no está validado contra libsrt y supone que el *edge* conoce sin error el instante de generación del ACK. Tercero, SSIM no reemplaza una prueba con observadores, y el umbral de 0,05 es nuestro; con GoP 30 y $\theta = 0{,}02$ la ventaja desaparece. Cuarto, el decodificador de FFmpeg oculta los errores a su manera; otro reproductor puede mostrar más o menos artefactos.

VII. Conclusiones y trabajo futuro

SRT no elimina el *macroblocking*: quedan los cortes más largos que $L - \mathrm{RTT} - d$. En esos cortes, el *I-header* que ya está en el *edge* acortó los artefactos visibles en un tercio con GoP 90 y en un 15 % con GoP 30, sin agregar latencia y con 0,7 % más de tasa de bits hacia el espectador. Para lograr lo mismo subiendo L, con cortes de 600 ms se necesitan 500 ms. El *edge* sabe a quién enviarlo sin cambiar SRT, por la marca de tiempo de los ACK. El costo está en la subida (+57 a +67 %), en un cuadro hasta 9 veces más grande al entregarlo y en una deriva leve. Con cortes cortos, SRT basta y el *I-header* no mejora; la propuesta sirve con GoP largos y cortes largos.

Este avance deja hechos el emulador en Linux y la comparación de los cuatro casos. Queda pendiente: (i) varias secuencias y trazas de pérdida reales; (ii) una política del *edge* que no envíe el *I-header* con cortes cortos, con QP alto o con el intra cerca; (iii) una prueba con observadores; (iv) validar el modelo contra libsrt sobre una red emulada.

Referencias

[1] M. P. Sharabayko, M. A. Sharabayko, J. Dube, J. S. Kim y J. W. Kim, "The SRT protocol," IETF Internet-Draft draft-sharabayko-srt-01, trabajo en curso, 2021. [En línea]. Disponible: `https://datatracker.ietf.org/doc/draft-sharabayko-srt/`. Accedido: 10 nov. 2026.

[2] C. Chen, W. Yin, Z. Huang y S. Shi, "AGiLE: Enhancing adaptive GOP in live video streaming," en *Proc. ACM Multimedia Systems Conf. (MMSys '24)*, 2024, pp. 34–44, doi: 10.1145/3625468.3647609.

[3] ITU-T, "Advanced video coding for generic audiovisual services," Recomendación ITU-T H.264, ago. 2024. [En línea]. Disponible: `https://www.itu.int/rec/T-REC-H.264`. Accedido: 10 nov. 2026.

[4] G. Van Wallendael, H. Mareen, J. Vounckx y P. Lambert, "Keyframe insertion: Enabling low-latency random access and packet loss repair," *Electronics*, vol. 10, no. 6, art. 748, 2021, doi: 10.3390/electronics10060748.

[5] H. Mareen, M. Courteaux, P.-J. Speelmans, P. Lambert y G. Van Wallendael, "A study on keyframe injection in three generations of video coding standards for fast channel switching and packet-loss repair," *Multimedia Tools and Applications*, vol. 83, pp. 44485–44501, 2024, doi: 10.1007/s11042-023-17163-2.

[6] H. Mareen, P. Lambert y G. Van Wallendael, "P-frame injection for efficient packet-loss repair in ultra-low-latency video streaming," en *Proc. IEEE Int. Conf. Visual Communications and Image Processing (VCIP)*, 2023. [En línea]. Disponible: `https://ieeexplore.ieee.org/document/10402720/`. Accedido: 10 nov. 2026.

[7] B. Denizer, L. Winkel y O. Landsiedel, "FrameTrace: Frame-level telemetry for Media over QUIC," en *Proc. EMS '25, ACM SIGCOMM 2025 Workshops*, 2025, pp. 31–36, doi: 10.1145/3746441.3748231.

[8] Haivision, "RTT multiplier," en *SRT Deployment Guide*, v1.5.3. [En línea]. Disponible: `https://doc.haivision.com/SRT/1.5.3/Haivision/rtt-multiplier`. Accedido: 10 nov. 2026.

[9] Z. Wang, A. C. Bovik, H. R. Sheikh y E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Processing*, vol. 13, no. 4, pp. 600–612, abr. 2004, doi: 10.1109/TIP.2003.819861.

[10] Xiph.org, "Xiph.org Video Test Media [derf's collection]." [En línea]. Disponible: `https://media.xiph.org/video/derf/`. Accedido: 10 nov. 2026.